

Angular velocity calibration

This lesson uses the Raspberry Pi as an example. For Raspberry Pi and Jetson-Nano boards, you need to open a terminal on the host computer and enter the command to enter the Docker container. Once inside the Docker container, enter the commands mentioned in this lesson in the terminal. For instructions on entering the Docker container from the host computer, refer to **[01.Robot Configuration and Operation Guide] -- [5.Enter Docker (For JETSON Nano and RPi 5)]**. For Orin and RDK boards, simply open the terminal and enter the commands mentioned in this lesson.

1. Program Description

Run the program and adjust the parameters using the dynamic parameter adjuster to calibrate the car's angular velocity. To visually demonstrate the angular velocity calibration, give the command to move the car forward 1 meter and observe the actual distance traveled, checking if it falls within the error range.

2. Starting the Program

2.1 Starting Commands

For the Raspberry Pi 5 and jetson nano controller, you must first enter the Docker container. For the Orin and RDK controller, this is not necessary.

Entering the Docker container (see [Docker Course] --- [4. Docker Startup Script] for steps).

All the following commands must be executed within the same Docker container in the Docker terminal.** (see [Docker Course] --- [3. Docker Commit and Multiple Terminal Entry] for steps.)

Start the chassis data, enter the command in the terminal.

```
ros2 launch yahboomcar_bringup yahboomcar_bringup_A1_launch.py
```

```
root@raspberrypi: /
File Edit Tabs Help
got segment base_link
[robot_state_publisher-2] [INFO] [1755157941.620678910] [robot_state_publisher]:
got segment camera_link
[robot_state_publisher-2] [INFO] [1755157941.620688336] [robot_state_publisher]:
got segment imu_link
[robot_state_publisher-2] [INFO] [1755157941.620698873] [robot_state_publisher]:
got segment laser
[robot_state_publisher-2] [INFO] [1755157941.620707650] [robot_state_publisher]:
got segment left_front_wheel_joint
[robot_state_publisher-2] [INFO] [1755157941.620717132] [robot_state_publisher]:
got segment left_rear_wheel_hinge
[robot_state_publisher-2] [INFO] [1755157941.620726391] [robot_state_publisher]:
got segment left_steering_hinge_joint
[robot_state_publisher-2] [INFO] [1755157941.620734965] [robot_state_publisher]:
got segment right_front_wheel_joint
[robot_state_publisher-2] [INFO] [1755157941.620745576] [robot_state_publisher]:
got segment right_rear_wheel_hinge
[robot_state_publisher-2] [INFO] [1755157941.620755039] [robot_state_publisher]:
got segment right_steering_hinge_joint
[joint_state_publisher-1] [INFO] [1755157942.125945703] [joint_state_publisher]:
Waiting for robot_description to be published on the robot_description topic...
[imu_filter_madgwick_node-5] [INFO] [1755157942.341251881] [imu_filter_madgwick]:
First IMU message received.
```

```
ros2 run yahboomcar_bringup calibrate_angular_A1
```

The following image is displayed successfully.

```
root@raspberrypi: /# ^C
root@raspberrypi: /# ros2 run yahboomcar_bringup calibrate_angular_A1
finish init work
[INFO] [1755159169.096285150] [calibrate_angular]: transform not ready
```

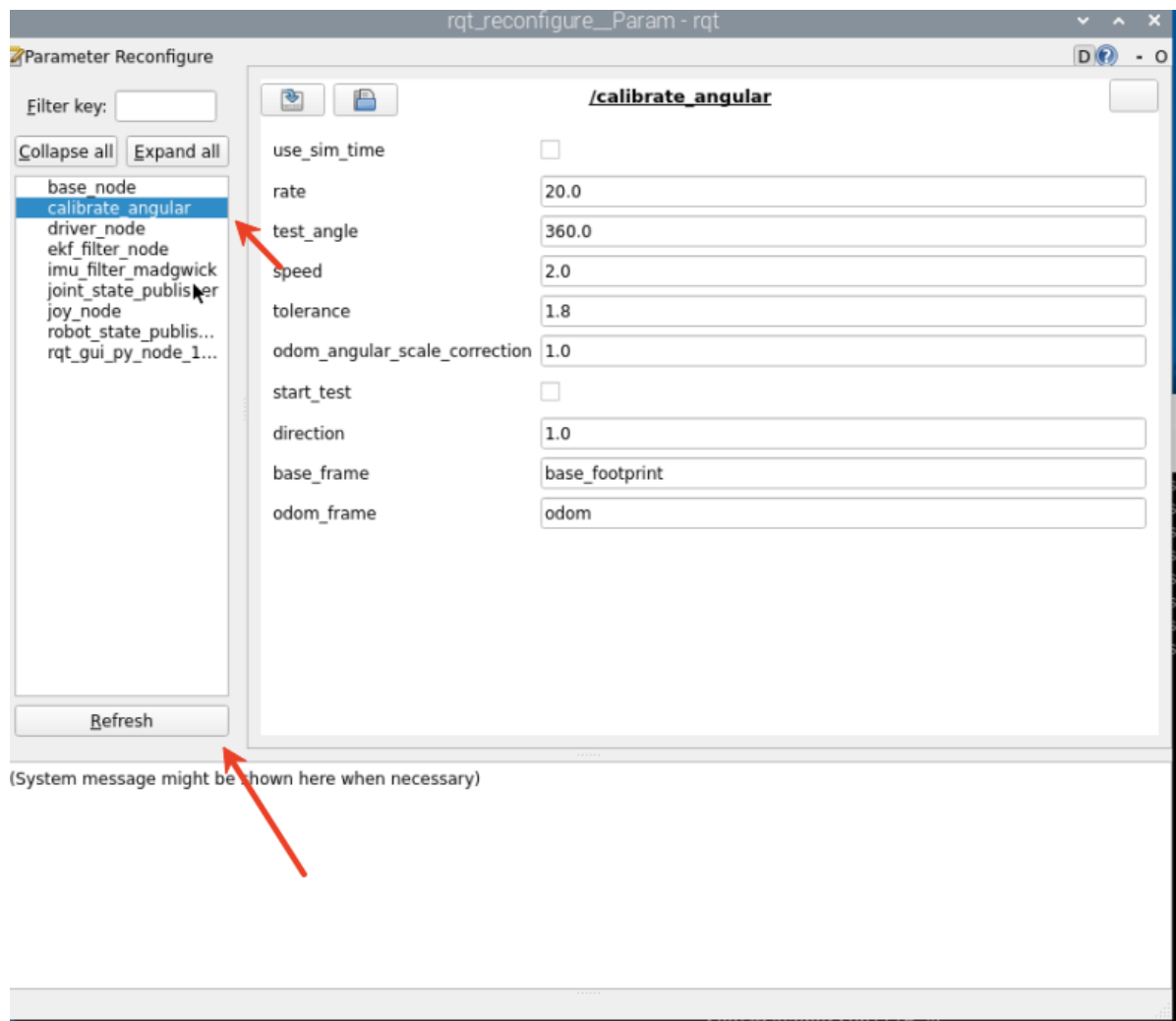
If the runtime error message indicates that no TF transformations are available, press **Ctrl+C** to exit the program and run it again.

```
File "/root/yahboomcar_ros2_ws/yahboomcar_ws/install/yahboomcar_bringup/lib/yahboomcar_bringup/calibrate_linear_A1", line 33, in <module>
  sys.exit(load_entry_point('yahboomcar-bringup==0.0.0', 'console_scripts', 'calibrate_linear_A1')())
File "/root/yahboomcar_ros2_ws/yahboomcar_ws/install/yahboomcar_bringup/lib/python3.10/site-packages/yahboomcar_bringup/calibrate_linear_A1.py", line 148, in main
  rclpy.spin(class_calibratelinear)
File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/__init__.py", line 226, in spin
  executor.spin_once()
File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/executors.py", line 751, in spin_once
  self._spin_once_impl(timeout_sec)
File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/executors.py", line 748, in _spin_once_impl
  raise handler.exception()
File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/task.py", line 254, in __call__
  self._handler.send(None)
File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/executors.py", line 447, in handler
  await call_coroutine(entity, arg)
File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/executors.py", line 361, in _execute_timer
  await await_or_execute(tmr.callback)
File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/executors.py", line 107, in await_or_execute
  return callback(*args)
File "/root/yahboomcar_ros2_ws/yahboomcar_ws/install/yahboomcar_bringup/lib/python3.10/site-packages/yahboomcar_bringup/calibrate_linear_A1.py", line 114, in on_timer
  self.x_start = self.get_position().transform.translation.x
File "/root/yahboomcar_ros2_ws/yahboomcar_ws/install/yahboomcar_bringup/lib/python3.10/site-packages/yahboomcar_bringup/calibrate_linear_A1.py", line 136, in get_position
  trans = self.tf_buffer.lookup_transform(self.odom_frame, self.base_frame, now)
File "/opt/ros/humble/lib/python3.10/site-packages/tf2_ros/buffer.py", line 136, in lookup_transform
  return self.lookup_transform_core(target_frame, source_frame, time)
tf2.LookupException: "odom" passed to lookupTransform argument target_frame does not exist.
root@raspberrypi:/#
```

Open the dynamic parameter adjuster and run the following command in the terminal:

```
ros2 run rqt_reconfigure rqt_reconfigure
```

Click the **calibrate_angular** node in the node options on the left:



Note: The above nodes may not appear when you first open the application. Click Refresh to see all nodes. The **calibrate_angular** node displayed is the node for calibrating angular velocity.

Other parameters in the rqt interface are described below:

- test_angle: The angle used for the calibration test. Here, a 360-degree rotation is tested.
- speed: The angular velocity.
- tolerance: The tolerance for error.
- odom_angular_scale_correction: The linear velocity scale factor. If the test results are unsatisfactory, adjust this value.
- start_test: Test on/off.
- base_frame: The name of the base coordinate system.
- odom_frame: The name of the odometry coordinate system.

2.2. Start Calibration

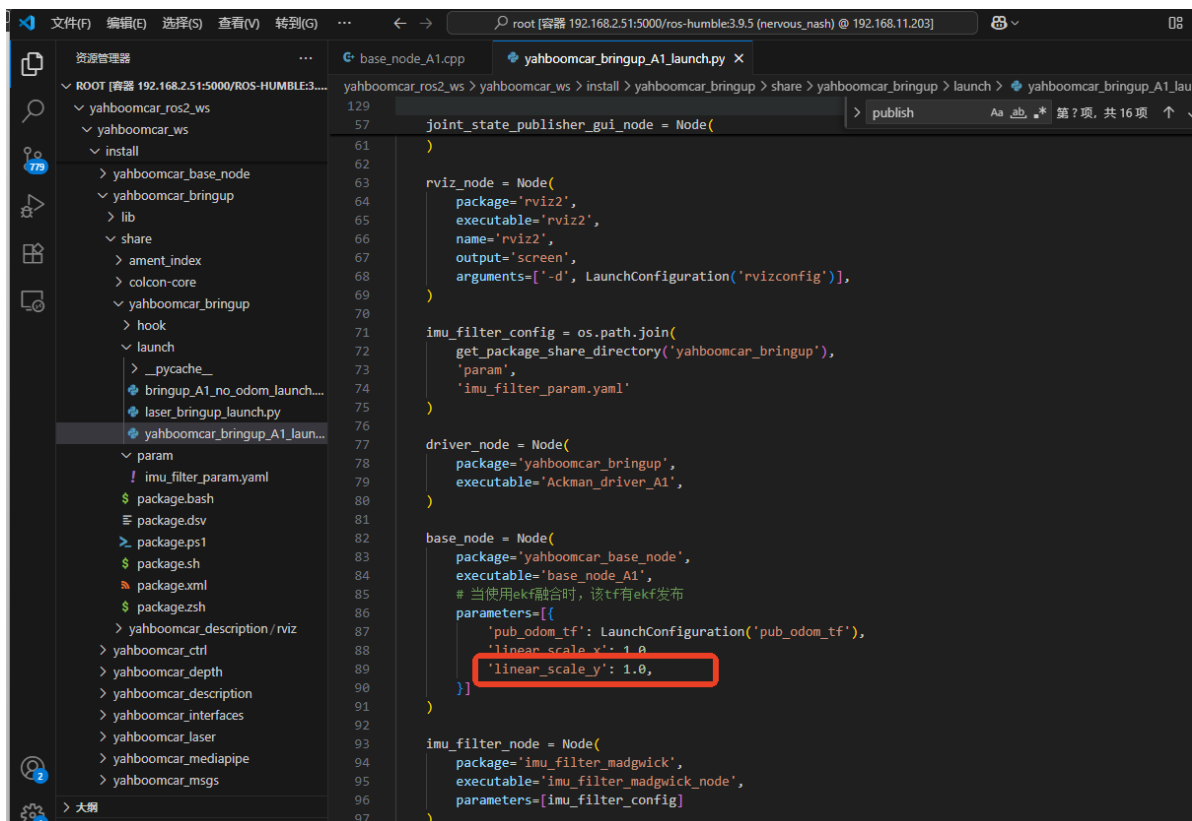
In the rqt_reconfigure interface, select the calibrate_angular node. Click the box to the right of the **start_test** node to begin calibration.

After clicking start_test, calibration begins. The car will monitor the TF transformations of base_footprint and odom, calculate the theoretical rotation angle, and issue a stop command when the error is less than the tolerance.

```
root@raspberrypi: /
File Edit Tabs Help

turn_angle: 4.456176502295412
error: 1.8270088048841746
turn_angle: 4.456176502295412
error: 1.8270088048841746
turn_angle: 4.456176502295412
error: 1.8270088048841746
turn_angle: 4.456176502295412
error: 1.8270088048841746
turn_angle: 4.456176502295412
error: 1.8270088048841746
turn_angle: 4.456176502295412
error: 1.8270088048841746
turn_angle: 4.456176502295412
error: 1.8270088048841746
turn_angle: 4.456176502295412
error: 1.8270088048841746
turn_angle: 4.456176502295412
error: 1.8270088048841746
turn_angle: 4.536311816279438
error: 1.746873490900148
done
█
```

After the test is complete, remember the value of [odom_angular_scale_correction] and modify it to the linear_scale_y parameter in yahboomcar_bringup_A1_launch.py.

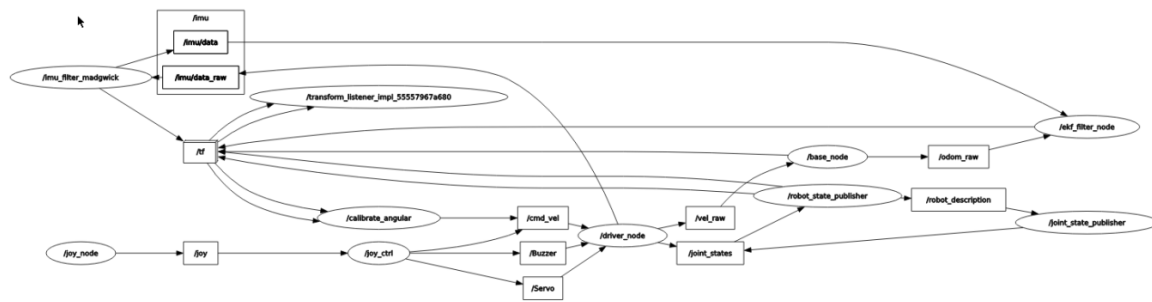


```
base_node_A1.cpp yahboomcar_bringup_A1_launch.py X
129
57 joint_state_publisher_gui_node = Node(
61 )
62
63 rviz_node = Node(
64     package='rviz2',
65     executable='rviz2',
66     name='rviz2',
67     output='screen',
68     arguments=['-d', LaunchConfiguration('rvizconfig')],
69 )
70
71 imu_filter_config = os.path.join(
72     get_package_share_directory('yahboomcar_bringup'),
73     'param',
74     'imu_filter_param.yaml'
75 )
76
77 driver_node = Node(
78     package='yahboomcar_bringup',
79     executable='Ackman_driver_A1',
80 )
81
82 base_node = Node(
83     package='yahboomcar_base_node',
84     executable='base_node_A1',
85     # 当使用ekf融合时, 该tf有ekf发布
86     parameters=[(
87         'pub_odom_tf': LaunchConfiguration('pub_odom_tf'),
88         'linear_scale_x': 1.0,
89         'linear_scale_y': 1.0,
90     )]
91 )
92
93 imu_filter_node = Node(
94     package='imu_filter_madgwick',
95     executable='imu_filter_madgwick_node',
96     parameters=[imu_filter_config]
97 )
```

3. View the Node Relationship Graph

Open a terminal and enter the command:

```
ros2 run rqt_graph rqt_graph
```



In the above node relationship graph:

- The **imu_filter** node is responsible for filtering the chassis's raw IMU data **/imu/data** and publishing the filtered data **/imu/data**.
- The **/ekf_filter_node** node subscribes to the chassis's raw odometry data **/odom_raw** and filtered IMU data **/imu/data**, performs data fusion, and publishes the data to the **/odom** topic.
- The **calibrate_angular** node monitors the TF transformation between odom->base_footprint and publishes the **/cmd_vel** topic to control the robot's chassis movement.

4. Core Source Code Analysis

This program primarily utilizes TF to monitor coordinate transformations. By monitoring the coordinate transformation between base_footprint and odom, the robot can determine "how far I've walked" or "how many degrees I've turned."

Code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/yahboomcar_bringup/calibrate_angular_A1.py
```

Among them, the implementation of monitoring tf coordinate transformation is the `get_odom_angle` method in the `Calibrateangular` class:

```
def get_odom_angle(self):
    try:
        now = rclpy.time.Time()
        rot = self.tf_buffer.lookup_transform(
            self.base_frame,
            self.odom_frame,
            now,
            timeout=rclpy.duration.Duration(seconds=1.0))
        cac1_rot = PyKDL.Rotation.Quaternion(rot.transform.rotation.x,
            rot.transform.rotation.y, rot.transform.rotation.z,
            rot.transform.rotation.w)
        #print("cac1_rot: ",cac1_rot)
        angle_rot = cac1_rot.GetRPY()[2]
        #print("angle_rot: ",angle_rot)

    except (LookupException, ConnectivityException, ExtrapolationException):
        # self.get_logger().info('transform not ready')
        return
```

The `on_timer` (timer callback function) method in the `Calibrateangular` class is used to determine the rotation angle of the robot chassis and control the chassis movement:

```

def on_timer(self):
    self.start_test =
self.get_parameter('start_test').get_parameter_value().bool_value
    self.odom_angular_scale_correction =
self.get_parameter('odom_angular_scale_correction').get_parameter_value().double_value
    self.test_angle =
self.get_parameter('test_angle').get_parameter_value().double_value
    self.test_angle = radians(self.test_angle) #角度转成弧度
    self.speed = self.get_parameter('speed').get_parameter_value().double_value
    move_cmd = Twist()
    self.test_angle *= self.reverse
    #self.test_angle *= self.reverse
    #self.error = self.test_angle - self.turn_angle
    if self.start_test:
        self.error = self.turn_angle - self.test_angle
        if abs(self.error) > self.tolerance :
            #move_cmd.linear.x = 0.2
            move_cmd.angular.z = copysign(self.speed, self.error)
            #print("angular: ",move_cmd.angular.z)
            self.cmd_vel.publish(move_cmd)
            self.odom_angle = self.get_odom_angle()
            self.delta_angle = self.odom_angular_scale_correction *
self.normalize_angle(self.odom_angle - self.first_angle)
            #print("delta_angle: ",self.delta_angle)
            self.turn_angle += self.delta_angle
            print("turn_angle: ",self.turn_angle,flush=True)
            #self.error = self.test_angle - self.turn_angle
            print("error: ",self.error,flush=True)
            self.first_angle = self.odom_angle

            #print("first_angle: ",self.first_angle)
        else:
            self.error = 0.0
            self.turn_angle = 0.0
            print("done",flush=True)
            self.first_angle = 0
            self.reverse = -self.reverse
            self.start_test =
rcipy.parameter.Parameter('start_test',rcipy.Parameter.Type.BOOL,False)
            all_new_parameters = [self.start_test]
            self.set_parameters(all_new_parameters)
    else:
        self.error = 0.0
        self.cmd_vel.publish(Twist())
        self.turn_angle = 0.0
        self.start_test =
rcipy.parameter.Parameter('start_test',rcipy.Parameter.Type.BOOL,False)

        all_new_parameters = [self.start_test]
        self.set_parameters(all_new_parameters)

```